Splunk: Manipulação de Dados

Gestão de Parsing e Extração de Campos (`props.conf`)

Objetivo

Demonstrar competência técnica no pipeline de processamento de dados do Splunk, focando na estruturação de logs brutos e criação de extrações de campos customizadas para otimizar buscas e correlações.

Ambiente Utilizado

- **Configuração:** Arquivo `props.conf` em `$SPLUNK_HOME/etc/system/local/`.
- **Conceitos:** Sourcetyping, Regex, Search-Time Extraction.

Procedimento Técnico Resumido

1. **Mapeamento de Sourcetype:** Identificação da fonte de dados e atribuição de uma categoria específica no `props.conf` para aplicar regras de tratamento.
2. **Configuração de Extração (Regex):** Implementação de expressões regulares para capturar dados não estruturados e transformá-los em campos indexáveis.
    - *Exemplo técnico:* `EXTRACT-user_id = userID=(?<user_id>\d+)`
3. **Validação:** Reinicialização do serviço e verificação via SPL para garantir que os novos campos (campos de interesse) estejam disponíveis para filtros e comandos estatísticos.

Resultado Obtido

Transformação de dados brutos em metadados estruturados, permitindo que a equipe de segurança execute consultas complexas e dashboards precisos sobre fontes de dados proprietárias.

Conclusão Técnica

O domínio do `props.conf` é essencial para a normalização de dados em um SIEM. Esta habilidade garante visibilidade total sobre logs que não seguem padrões de mercado, aumentando a capacidade de detecção de ameaças e a eficiência na resposta a incidentes.

Gestão de Arquivos de Configuração e Stanzas no Splunk

Objetivo

Demonstrar conhecimento avançado na administração do Splunk através da manipulação de arquivos de configuração (.conf). O foco é entender como controlar o fluxo de dados, desde a coleta e processamento até o armazenamento e encaminhamento, utilizando *stanzas* para parametrização granular.

Ambiente Utilizado

- **Arquitetura:** Splunk Enterprise / Universal Forwarder.
- **Diretório de Configuração:** $SPLUNK_HOME/etc/system/local/ ou instâncias de *Apps*.
- **Arquivos Chave:** inputs.conf, props.conf, transforms.conf, indexes.conf, outputs.conf.

Procedimento Técnico Resumido

1. Controle do Ciclo de Vida dos Dados

A administração foi estruturada através dos principais arquivos de configuração:

- **inputs.conf**: Define **o que** coletar (arquivos, portas TCP/UDP).
- **props.conf**: Define **como** interpretar os dados (quebra de linha, timestamps, extrações).
- **transforms.conf**: Define **mudanças** nos dados (mascaramento, roteamento para índices específicos).
- **indexes.conf**: Define **onde e por quanto tempo** armazenar (retenção, caminhos de storage).
- **outputs.conf**: Define **para onde** enviar (encaminhamento para Indexers).

2. Configuração via Stanzas (Processamento de Eventos)

Para garantir a integridade dos logs, foram aplicadas configurações específicas dentro do props.conf:

- **Tratamento de Linhas:** Uso de SHOULD_LINEMERGE = false e LINE_BREAKER para evitar que múltiplos eventos sejam fundidos incorretamente.
- **Normalização de Tempo:** Aplicação de TIME_FORMAT e TIME_PREFIX para garantir a precisão da linha do tempo nas investigações.
- **Estruturação:** Uso de KV_MODE = json para parsing automático de campos em logs estruturados.

Resultado Obtido

Configuração de um ambiente Splunk altamente customizado e performático. A correta manipulação dos arquivos .conf permite a ingestão de fontes de dados complexas, garante a precisão dos timestamps e otimiza o armazenamento, resultando em buscas mais rápidas e alertas mais confiáveis.

Conclusão Técnica

O domínio dos arquivos de configuração e suas *stanzas* é a base da engenharia de dados no Splunk. Esta competência permite que o analista vá além da interface gráfica, resolvendo problemas complexos de *parsing* e arquitetura, garantindo que o SIEM opere com máxima fidelidade aos dados brutos e conformidade com as políticas de retenção da organização.

Desenvolvimento de Aplicação Customizada e Ingestão de Dados via Script

Objetivo

Demonstrar a capacidade de estender a funcionalidade do Splunk através da criação de um **Splunk App** customizado. O procedimento foca na automação da ingestão de dados através de um script Python (`inputs.conf` tipo script), estruturando o diretório da aplicação e configurando a coleta periódica de logs de exemplo para validação da pipeline de dados.

Ambiente Utilizado

- **Ferramenta:** Splunk Enterprise (CLI e Interface Web).
- **Diretório Base:** `/opt/splunk/`.
- **Linguagem:** Python 3 (para geração de logs).

Procedimento Técnico Resumido

1. Criação do Aplicativo (App)

- **Ação:** Acessado `Manage Apps` > `Create App` via interface web.
- **Estrutura:** O app foi criado no diretório `/opt/splunk/etc/apps/DataApp`, gerando a estrutura padrão: `bin/`, `default/`, `local/` e `metadata/`.

2. Desenvolvimento do Gerador de Logs

- **Ação:** Criado um script Python (`samplelogs.py`) dentro do diretório `bin/` da aplicação para gerar logs sintéticos.
- **Conteúdo do Script:** `print("This is a sample log...")`

3. Configuração de Ingestão (`inputs.conf`)

- **Ação:** Criado o arquivo `inputs.conf` no diretório `default/` para instruir o Splunk a executar o script e indexar a saída.
- **Configuração Técnica:**

```
[script:///opt/splunk/etc/apps/DataApp/bin/samplelogs.py]
index = main
source = test_log
```

```
sourcetype = testing
interval = 5
```

- **Explicação:** O script é executado como um usuário do sistema, enviando a saída (`stdout`) para o índice `main` a cada 5 segundos.

4. Aplicação e Validação

- **Ação:** Reiniciado o serviço Splunk (`/opt/splunk/bin/splunk restart`).
- **Verificação:** Acessada a interface `Search & Reporting` para confirmar a ingestão dos dados.

Resultado Obtido

Uma nova aplicação funcional foi implementada, e logs de teste estão sendo gerados automaticamente pelo script Python e indexados pelo Splunk. Isso confirma a correta integração entre o gerador de dados, a configuração de inputs e a plataforma de análise.

Conclusão Técnica

A criação de aplicações customizadas é essencial para adaptar o Splunk a ambientes com fontes de dados proprietárias. Esta tarefa demonstra habilidades em **administração de Linux**, **automação básica em Python** e **engenharia de dados no Splunk**, permitindo a criação de *pipelines* de dados flexíveis e robustos.

Configuração de Parsing e Divisão de Eventos (`props.conf`)

Objetivo

Demonstrar a capacidade técnica de solucionar problemas de ingestão de dados no Splunk, onde múltiplos eventos são incorretamente indexados como um único evento. O foco é configurar regras de `props.conf` para identificar corretamente os limites de início e fim de cada log, garantindo a integridade dos dados para análise.

Ambiente Utilizado

- **Ferramenta:** Splunk Enterprise (CLI e Interface Web).
- **Diretório:** `/opt/splunk/etc/apps/DataApp/`.
- **Arquivos:** `inputs.conf` (coleta) e `props.conf` (parsing).

Procedimento Técnico Resumido

1. Configuração da Coleta de Dados (`inputs.conf`)

- **Ação:** Adicionado o script Python `vpnlogs` à aplicação `DataApp` para gerar logs sintéticos de VPN.
- **Configuração:**

```
[script:///opt/splunk/etc/apps/DataApp/bin/vpnlogs]
index = main
source = vpn
sourcetype = vpn_logs
interval = 5
```

2. Identificação do Problema de Parsing

- **Ação:** Validação na interface de busca (`index=main sourcetype=vpn_logs`).
- **Problema:** O Splunk não reconhece as quebras de evento corretamente, agrupando múltiplas linhas em um único registro indexado.

3. Correção via `props.conf`

- **Ação:** Criação/Edição do arquivo `props.conf` em `default/` para definir regras de quebra de evento (Event Breaking).
- **Configuração Técnica:**

```
[vpn_logs]
SHOULD_LINEMERGE = true
MUST_BREAK_AFTER = (DISCONNECT|CONNECT)
```

- **Explicação:**
  - `SHOULD_LINEMERGE = true`: Indica ao Splunk para mesclar linhas antes de aplicar regras de quebra.
  - `MUST_BREAK_AFTER = (DISCONNECT|CONNECT)`: Utiliza Regex para forçar o fim do evento logo após as palavras-chave "CONNECT" ou "DISCONNECT".

[INSERIR IMAGEM: Print da interface de busca do Splunk ANTES da correção, mostrando múltiplos logs em um único evento]

Legenda: Figura 1 - Logs de VPN incorretamente agrupados como um único evento.

[INSERIR IMAGEM: Print da interface de busca do Splunk DEPOIS da correção, mostrando eventos individuais corretamente divididos]

Legenda: Figura 2 - Logs de VPN corretamente divididos após configuração do MUST_BREAK_AFTER.

Resultado Obtido

Após reiniciar o serviço Splunk, os eventos passaram a ser divididos corretamente. Cada log de conexão/desconexão VPN foi indexado como um evento individual, permitindo pesquisas precisas, contagens estatísticas exatas e a criação de alertas confiáveis.

Conclusão Técnica

A correta configuração do `props.conf` é fundamental para a integridade de um SIEM. Esta tarefa demonstra habilidades avançadas em **engenharia de dados no Splunk**, uso de **expressões regulares (Regex)** para parsing e capacidade de *troubleshooting* em pipelines de ingestão de logs customizados.

Configuração de Parsing de Eventos Multilinha (props.conf)

Objetivo

Demonstrar a capacidade técnica de solucionar problemas de ingestão de dados no Splunk, onde eventos compostos por múltiplas linhas são incorretamente indexados como eventos separados. O foco é configurar regras de props.conf para identificar o início exato de um evento, garantindo a integridade dos dados para análise.

Ambiente Utilizado

- **Ferramenta:** Splunk Enterprise (CLI e Interface Web).
- **Diretório:** /opt/splunk/etc/apps/DataApp/.
- **Arquivos:** inputs.conf (coleta) e props.conf (parsing).

Procedimento Técnico Resumido

1. Configuração da Coleta de Dados (inputs.conf)

- **Ação:** Adicionado o script Python authentication_logs à aplicação DataApp para gerar logs sintéticos multilinha.
- **Configuração:**
- Ini, TOML

```
[script:///opt/splunk/etc/apps/DataApp/bin/authentication_logs]
interval = 5
index = main
sourcetype= auth_logs
host = auth_server
```

-
-

2. Identificação do Problema de Parsing

- **Ação:** Validação na interface de busca (`index=main sourcetype=auth_logs`).
- **Problema:** O Splunk utiliza o padrão de quebra de linha como fim de evento, dividindo um log lógico de duas linhas em dois eventos físicos distintos.

3. Correção via `props.conf`

- **Ação:** Edição do arquivo `props.conf` em `default/` para definir regras de início de evento (Event Breaking).
- **Configuração Técnica:**
- Ini, TOML

```
[auth_logs]
SHOULD_LINEMERGE = true
BREAK_ONLY_BEFORE = \[Authentication\]
```

-
-
- **Explicação:**
  - `SHOULD_LINEMERGE = true`: Indica ao Splunk para mesclar linhas antes de aplicar regras de quebra.
  - `BREAK_ONLY_BEFORE = \[Authentication\]`: Utiliza Regex para identificar o termo `[Authentication]` como o início de um novo evento, mesclando tudo o que vier antes dele ao evento anterior.

[INSERIR IMAGEM: Print da interface de busca do Splunk ANTES da correção, mostrando logs multilinha divididos em dois eventos separados]

Legenda: Figura 1 - Logs multilinha incorretamente divididos em eventos distintos.

[INSERIR IMAGEM: Print da interface de busca do Splunk DEPOIS da correção, mostrando o evento multilinha consolidado corretamente]

Legenda: Figura 2 - Log multilinha corretamente consolidado após configuração do BREAK_ONLY_BEFORE.

Resultado Obtido

Após reiniciar o serviço Splunk, os eventos multilinha passaram a ser corretamente consolidados. O termo `[Authentication]` passou a atuar como âncora de início de evento, permitindo que logs complexos (similares a logs do Windows) sejam analisados de forma estruturada e íntegra.

Conclusão Técnica

A correta configuração do `props.conf` é fundamental para a integridade de um SIEM. Esta tarefa demonstra habilidades avançadas em **engenharia de dados no Splunk**, uso de **expressões regulares (Regex)** para controle de fluxo de parsing e capacidade de *troubleshooting* em pipelines de ingestão de logs complexos.

Documentação Técnica: Mascaramento de Dados Sensíveis e Otimização de Parsing no Splunk

1. Objetivo

Demonstrar a capacidade de implementar controles de segurança no Splunk para proteger dados sensíveis, garantindo a conformidade com normas como **PCI DSS** e **HIPAA**. O procedimento envolve a ingestão de logs de compras, correção da divisão de eventos físicos e o mascaramento (anonimização) de números de cartão de crédito durante o tempo de indexação.

2. Ambiente Utilizado

- **Ferramenta:** Splunk Enterprise (CLI e Interface Web).
- **Aplicação:** `DataApp` (Localizado em `/opt/splunk/etc/apps/DataApp/`).
- **Arquivos:** `inputs.conf` (coleta), `props.conf` (parsing e mascaramento).

3. Procedimento Técnico Resumido

3.1. Ingestão de Dados (`inputs.conf`)

- **Ação:** Configuração da ingestão do script `purchase-details` localizado no diretório `bin` da aplicação para gerar logs de transações.

Configuração Técnica:
Ini, TOML
[script:///opt/splunk/etc/apps/DataApp/bin/purchase-details]
interval = 5
index = main
source = purchase_logs
sourcetype= purchase_logs
host = order_server

- **[INSERIR IMAGEM: Print do arquivo inputs.conf configurado]** *Legenda: Figura 1 - Configuração de inputs.conf para coleta de logs de compra.*

3.2. Identificação de Problemas de Segurança e Parsing

- **Ação:** Pesquisa na interface `Search & Reporting` (`index=main sourcetype=purchase_logs`).
- **Problemas Identificados:**
  1. **Exposição de Dados:** Números de cartão de crédito exibidos em texto claro (cleartext), violando políticas de conformidade.

2. **Parsing Incorreto:** Múltiplas transações sendo agrupadas em um único evento lógico.

[INSERIR IMAGEM: Print da interface de busca mostrando os logs brutos com números de cartão expostos e agrupamento incorreto] *Legenda: Figura 2 - Logs brutos com dados sensíveis expostos e quebra de eventos incorreta.*

3.3. Correção da Divisão de Eventos (`props.conf`)

- **Ação:** Edição do arquivo `props.conf` para forçar o Splunk a entender onde cada transação termina.

Configuração Técnica:
Ini, TOML
[purchase_logs]
SHOULD_LINEMERGE = true
MUST_BREAK_AFTER = \.

-
- **Explicação:** O regex `\.` indica que o evento deve ser quebrado logo após o ponto final da frase.

3.4. Mascaramento de Dados Sensíveis (`SEDCMD`)

- **Ação:** Implementação de mascaramento via `SEDCMD` no `props.conf` para substituir os números de cartão de crédito por um formato mascarado (`XXXX-XXXX-XXXX`) antes da indexação.

Configuração Técnica:
Ini, TOML
[purchase_logs]
SHOULD_LINEMERGE = true
MUST_BREAK_AFTER = \.
SEDCMD-maskCC = s/-\d{4}-\d{4}-\d{4}/-XXXX-XXXX-XXXX/g

-
- **Explicação:** O `SEDCMD` utiliza sintaxe `sed` para localizar o padrão de 12 dígitos do cartão (após o primeiro bloco) e substituí-lo globalmente (g) por máscaras.

[INSERIR IMAGEM: Print do arquivo props.conf configurado com o MUST_BREAK_AFTER e o SEDCMD] *Legenda: Figura 3 - Configuração de props.conf para quebra de eventos e mascaramento de dados.*

3.5. Validação das Alterações

- **Ação:** Reinicialização do serviço Splunk e verificação na interface de busca.

[INSERIR IMAGEM: Print da interface de busca mostrando os logs com os números de cartão mascarados e eventos individuais] *Legenda: Figura 4 - Resultado final: eventos corretamente divididos e dados sensíveis mascarados.*

4. Resultado Obtido

A pipeline de dados foi ajustada com sucesso. Os eventos passaram a ser divididos corretamente e, mais importante, os números de cartão de crédito foram anonimizados no momento da indexação. Isso garante que analistas de SOC possam trabalhar com os logs sem acessar dados bancários confidenciais.

5. Conclusão Técnica

O mascaramento de dados no momento da ingestão é um controle crítico de segurança. Esta tarefa demonstra habilidades avançadas em **engenharia de dados no Splunk**, aplicação de **expressões regulares (Regex)** para substituição de strings e conhecimento prático de requisitos de **conformidade (PCI DSS/HIPAA)** na administração de SIEM.

Extração de Campos Customizados (`transforms.conf` e `fields.conf`)

Objetivo

Demonstrar a capacidade técnica de estruturar logs não estruturados no Splunk, transformando dados brutos em campos pesquisáveis (`Field Extraction`). Este procedimento é crucial para um analista de SOC, permitindo a criação de correlações, estatísticas e painéis baseados em informações específicas contidas no corpo do log (ex: usuário, servidor, ação).

Ambiente Utilizado

- **Ferramenta:** Splunk Enterprise (CLI e Interface Web).
- **Diretório:** `/opt/splunk/etc/apps/DataApp/`.
- **Arquivos:** `props.conf`, `transforms.conf`, `fields.conf`.

Procedimento Técnico Resumido

1. Identificação do Problema de Estruturação

- **Ação:** Validação na interface de busca (`index=main sourcetype=vpn_logs`).
- **Problema:** Os logs de VPN estão em texto claro e o Splunk não extrai campos como `username`, `server` e `action` automaticamente, impossibilitando análises estatísticas.

2. Extração de Campos via `transforms.conf`

- **Ação:** Criação do arquivo `transforms.conf` no diretório `default/` para definir a regra de extração utilizando Expressões Regulares (Regex).

Configuração Técnica (Multi-campo):
Ini, TOML
[vpn_custom_fields]

REGEX = User:\s([\w\s]+),\sServer:\s(\w+\s\w+),\sAction:\s(\w+)

FORMAT = Username::$1 Server::$2 Action::$3

WRITE_META = true

-
    - *Explicação:* O `REGEX` identifica os padrões no log e captura os valores em grupos (`$1`, `$2`, `$3`), mapeando-os para os nomes de campos definidos no `FORMAT`.

3. Vinculação no `props.conf`

- **Ação:** Atualização do `props.conf` para vincular o `sourcetype` à transformação criada.

Configuração Técnica:
Ini, TOML
[vpn_logs]

TRANSFORMS-vpn = vpn_custom_fields

-

4. Indexação de Campos (`fields.conf`)

- **Ação:** Criação do `fields.conf` para instruir o Splunk a extrair esses campos no momento da indexação (`Indexed Fields`), otimizando a performance das buscas.

Configuração Técnica:
Ini, TOML
[Username]

INDEXED = true

[Server]

INDEXED = true

[Action]

INDEXED = true

-

  [INSERIR IMAGEM: Print da interface de busca do Splunk ANTES da extração, mostrando apenas o campo _raw] *Legenda: Figura 1 - Dados brutos sem campos estruturados.*

[INSERIR IMAGEM: Print da interface de busca do Splunk DEPOIS da extração, mostrando os campos Username, Server e Action extraídos] *Legenda: Figura 2 - Dados estruturados com campos de interesse extraídos.*

Resultado Obtido

Após reiniciar o serviço Splunk, os campos `Username`, `Server` e `Action` tornaram-se campos indexados. Isso permite a execução de pesquisas complexas, como `index=main sourcetype=vpn_logs Action=CONNECT | stats count by Username`, facilitando a análise de comportamento e resposta a incidentes.

Conclusão Técnica

A habilidade de estruturar dados brutos é fundamental para a operação de um SIEM. Esta tarefa demonstra habilidades avançadas em **engenharia de dados no Splunk**, uso de **expressões regulares (Regex)** complexas e administração de arquivos de configuração, garantindo que o SOC tenha visibilidade total sobre fontes de dados proprietárias.